AI Agent Tool-Use Security Audit

Sample report — github.com/Semantic-Sentry/agent-tool-audit

What this measures: not whether your agent's reply looks safe, but whether untrusted input (a document, a customer message, a tool's own output) can make the agent actually **call a tool** with data it shouldn't send — confirmed by an independent receiver, not by trusting the model's own claim.

Headline finding: the model you choose changes your risk

On an **identical agent, identical prompt, identical attack vector** — only the model was swapped:

Model	Input reaching an outbound tool call	N
gpt-6-luna	**5.0%** (95% CI 0.9–23.6%)	20
deepseek-v4.1-flash	**55.0%** (95% CI 34.2–74.2%)	20

Non-overlapping confidence intervals — an **11× difference**. Teams that downshift to a cheaper model to save on tokens can change their security profile dramatically, and a response-level test would never catch it.

Every finding is graded and evidence-backed

Vector class	Model	Result	Outcome
Tool-call echo	gpt-6-luna	1/20 (5%)	Low / Integrity
Tool-call echo	deepseek-v4.1-flash	11/20 (55%)	Low / Integrity
Cross-customer leak	gpt-6-luna	0/5	Blocked (control)
Legacy 2024 framings	both models	0/50	Blocked (control)

Outcomes graded **NONE → ATTEMPT → UNBLOCKED → CONFIRMED**, where CONFIRMED means a canary value was physically captured by an independent sink — not inferred from the model's reply.

Negative controls matter: I report what your agent resists, not just what breaks. A clean result is documented, reproducible evidence — useful before you ship or change models.

How I work

1. **Map** the agent’s architecture, tools, and trust boundaries.
2. **Agree** an access policy — which tools are safe, egress, or forbidden on untrusted input.
3. **Test** reproducibly in an isolated sandbox with mock tools. Your production environment is never touched.
4. **Verify** outcomes independently — a canary captured by a receiver I control, not the model’s self-report.
5. **Deliver** a hash-anchored report: a config hash before the run, a manifest hash after, so the methodology is fixed before results exist and results can’t be silently edited.

You get: evidence report with reproduction steps, severity classification, statistical confidence intervals, remediation guidance, and (full tier) a pytest regression suite your team runs in CI before every prompt, tool, or model change.

Full methodology and independently verifiable sample evidence: github.com/Semantic-Sentry/agent-tool-audit